



LSTM

Understanding Long Short-Term Memory Networks

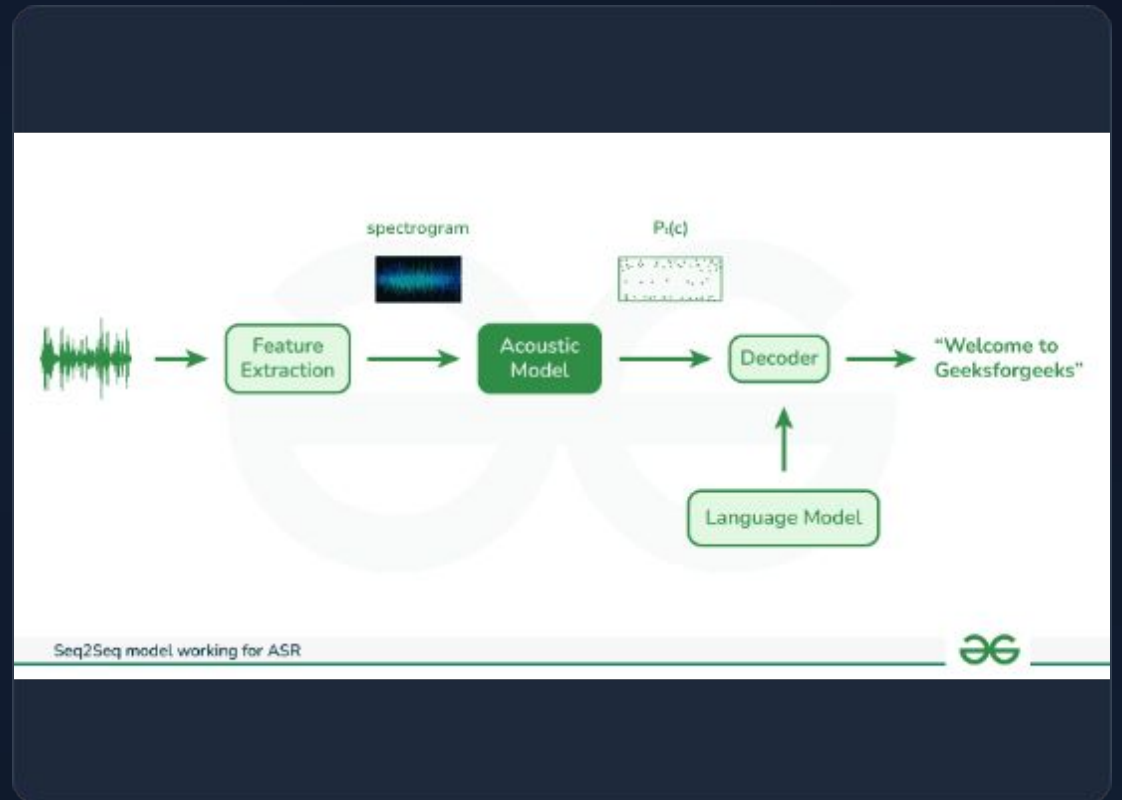
A Deep Dive into Sequence Modeling

1. Introduction: What is Sequence Data?

Not Just Static Points

Standard neural networks assume inputs are independent. However, in the real world, **order matters**.

- **Time Series:** Stock prices, weather, heartbeats.
- **Text (NLP):** "The cat sat on the..." (Context is key).
- **Audio:** Sound waves are sequential signals.



2. The Need for Memory



The Challenge

Humans don't start their thinking from scratch every second. As you read this sentence, you understand each word based on your understanding of previous words.



Persistence

Traditional Feed-Forward Neural Networks (FFNNs) cannot do this. They have no "memory" of the past. This is a major shortcoming for sequential tasks.

3. Recurrent Neural Networks (RNNs)

RNNs were the first solution to this problem.

They are networks with **loops** in them, allowing information to persist.

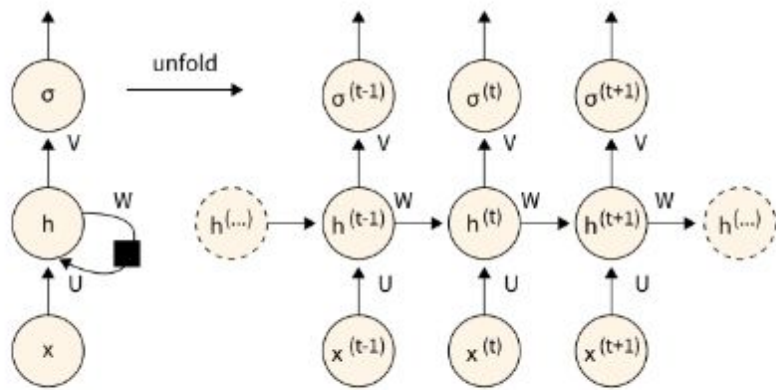
$$h_t = \tanh(W_h h_{t-1} + W_x x_t)$$

At step t , the network takes the current input and the previous hidden state . h_{t-1}



The output of the previous step becomes the input for the current step.

Visualizing the Loop



Unrolling the Network

An RNN can be thought of as multiple copies of the same network, each passing a message to a successor.

- **Inputs:** $x_0, x_1, x_2 \dots$
- **Hidden States:** $h_0, h_1, h_2 \dots$
- **Outputs:** $y_0, y_1, y_2 \dots$

4. The Problem: Short-Term Memory

"The Clouds are in the..."

If the gap between the relevant information and the point where it is needed is small, RNNs can learn to use the past information.

"I grew up in France... [long text] ... I speak..."

As the gap grows, RNNs become unable to learn to connect the information. The context of "France" is lost by the time we need to predict "French".

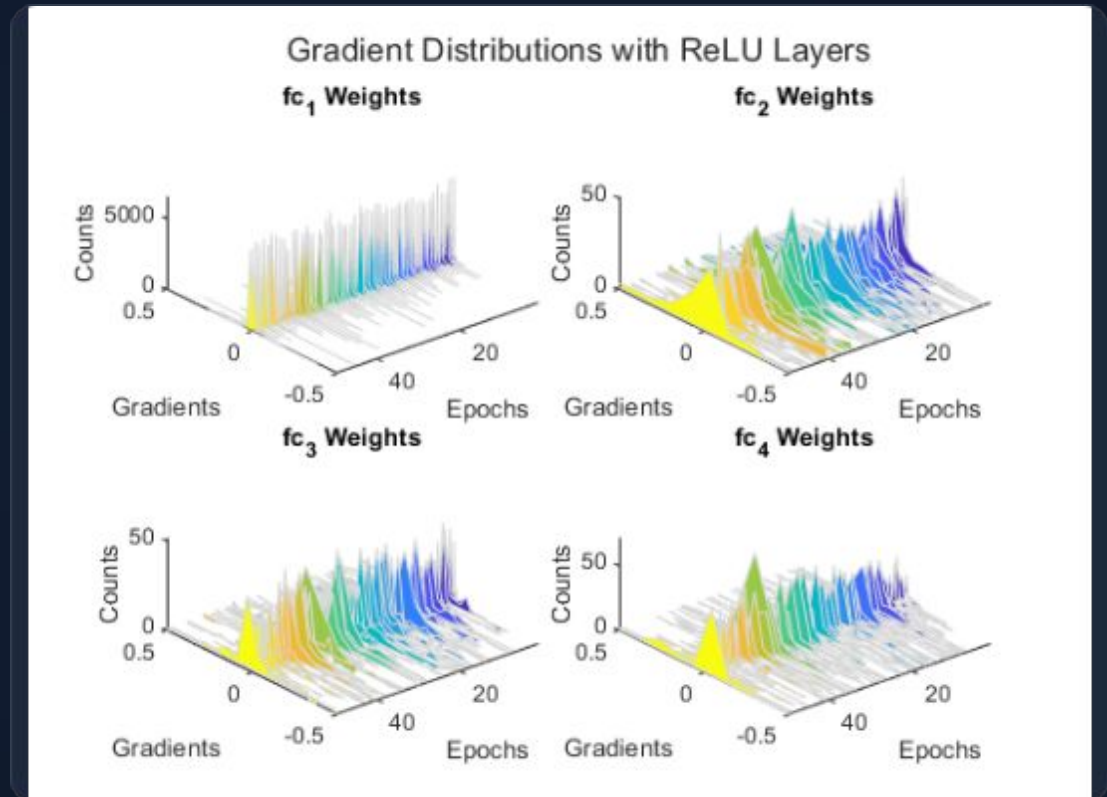
Why does this happen?

5. The Vanishing Gradient Problem

During backpropagation, gradients are used to update the network weights. In deep networks (or long sequences), gradients are multiplied repeatedly.

- If gradient < 1 : It shrinks exponentially to zero (Vanishes).
- If gradient > 1 : It grows exponentially (Explodes).

Result: The network stops learning from early layers.



6. Enter LSTM

Long Short-Term Memory

Proposed by **Hochreiter & Schmidhuber (1997)**

"LSTMs are explicitly designed to avoid the long-term dependency problem."

Remembering information for long periods of time is practically their default behavior.

Architecture Overview

All recurrent neural networks have the form of a chain of repeating modules of neural network.

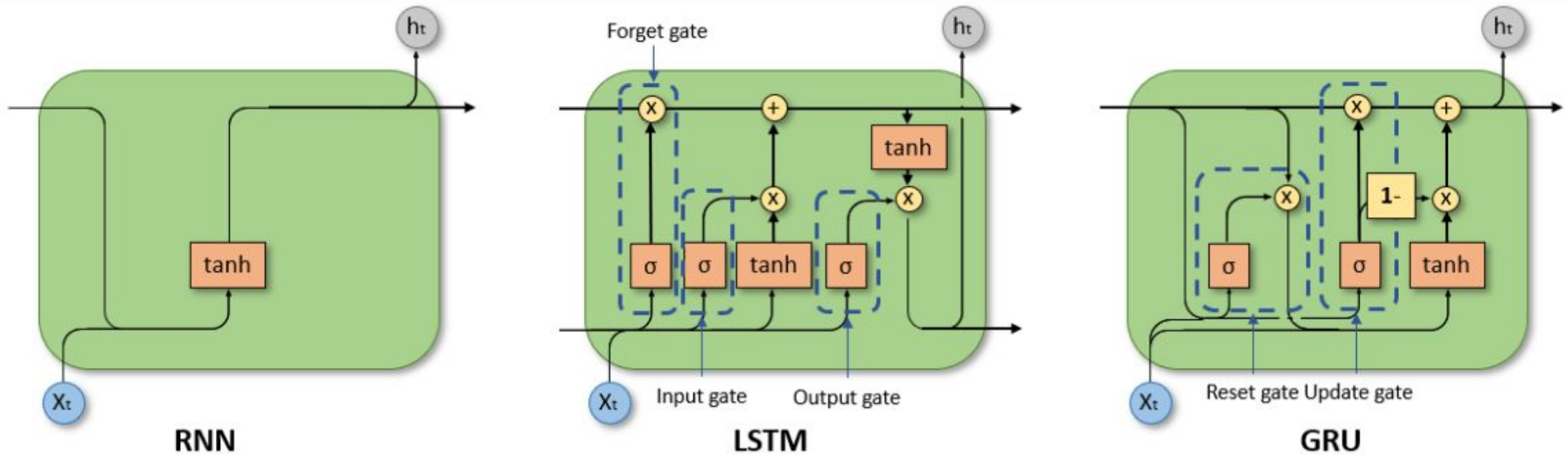
Standard RNN

The repeating module has a very simple structure, such as a single tanh layer.

LSTM

The repeating module has a different structure. Instead of having a single neural network layer, there are **four**, interacting in a very special way.

Standard RNN vs. LSTM



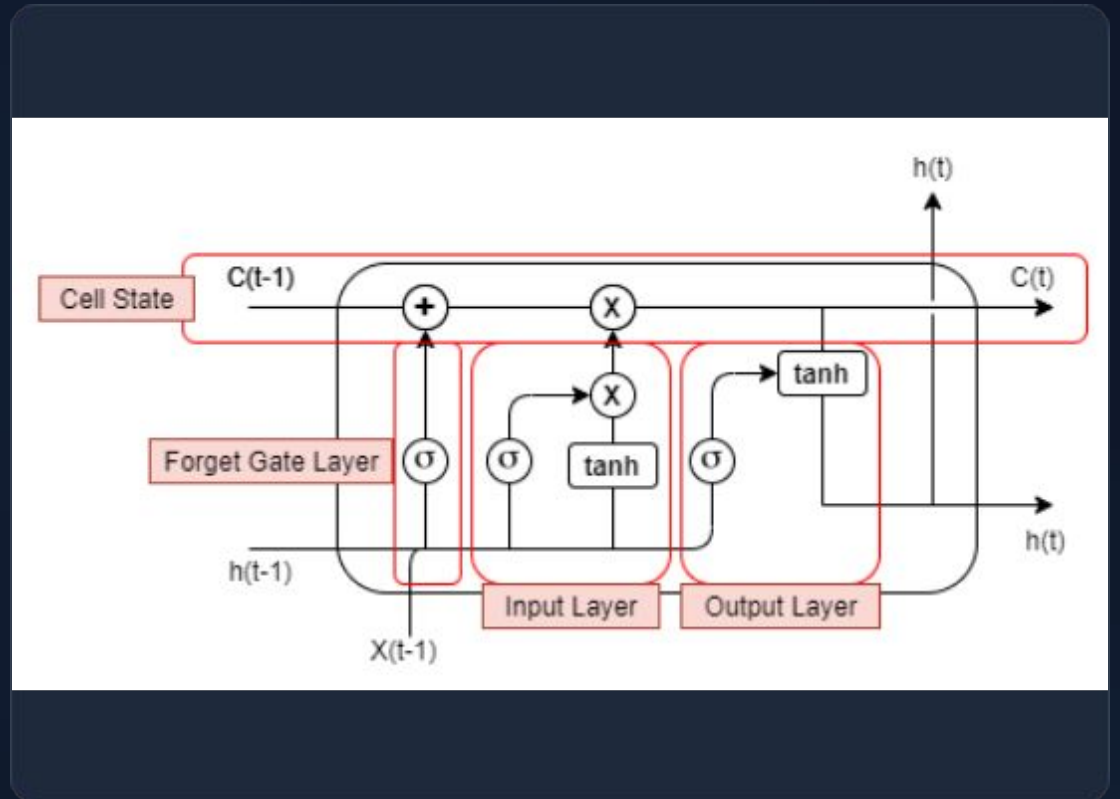
Notice the complexity inside the LSTM block compared to the simple RNN.

7. The Core Concept: Cell State

The Superhighway

The **Cell State** (c_t) is the horizontal line running through the top of the diagram.

It's like a conveyor belt. It runs straight down the entire chain, with only some minor linear interactions. It's very easy for information to flow along it unchanged.



8. Controlling Flow with Gates

Forget Gate

Decides what information to throw away from the cell state.



Input Gate

Decides what new information to store in the cell state.



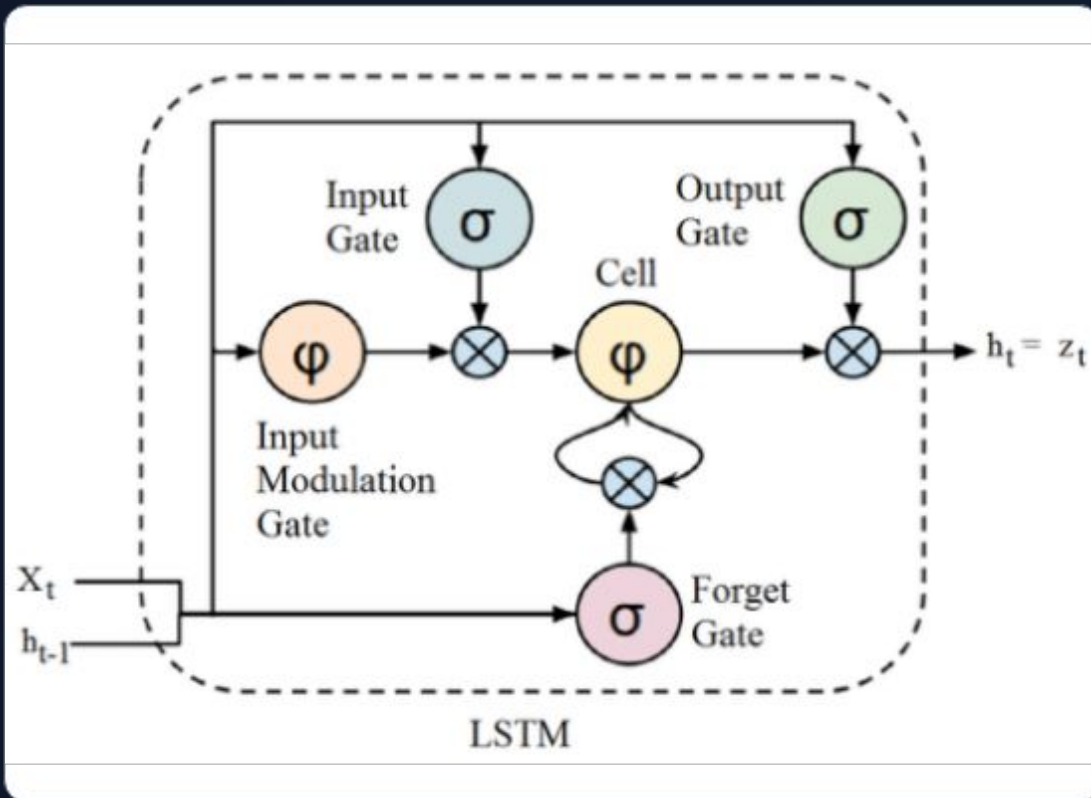
Output Gate

Decides what the next hidden state should be.



Gates are composed of a **sigmoid neural net layer** and a pointwise multiplication operation.

Step 1: The Forget Gate



"What do we keep?"

It looks at (previous hidden state) and (current input), and outputs a number between **0 and 1** for each number in the cell state .

c_{t-1}

- **1**: "Completely keep this."
- **0**: "Completely get rid of this."

Forget Gate Equation

$$f_t = \sigma W_f \cdot h_{t-1} x_t + b_f$$

f_t: The forget gate vector.

W_f: Weight matrix.

σ: Sigmoid function (squashes values to 0-1).

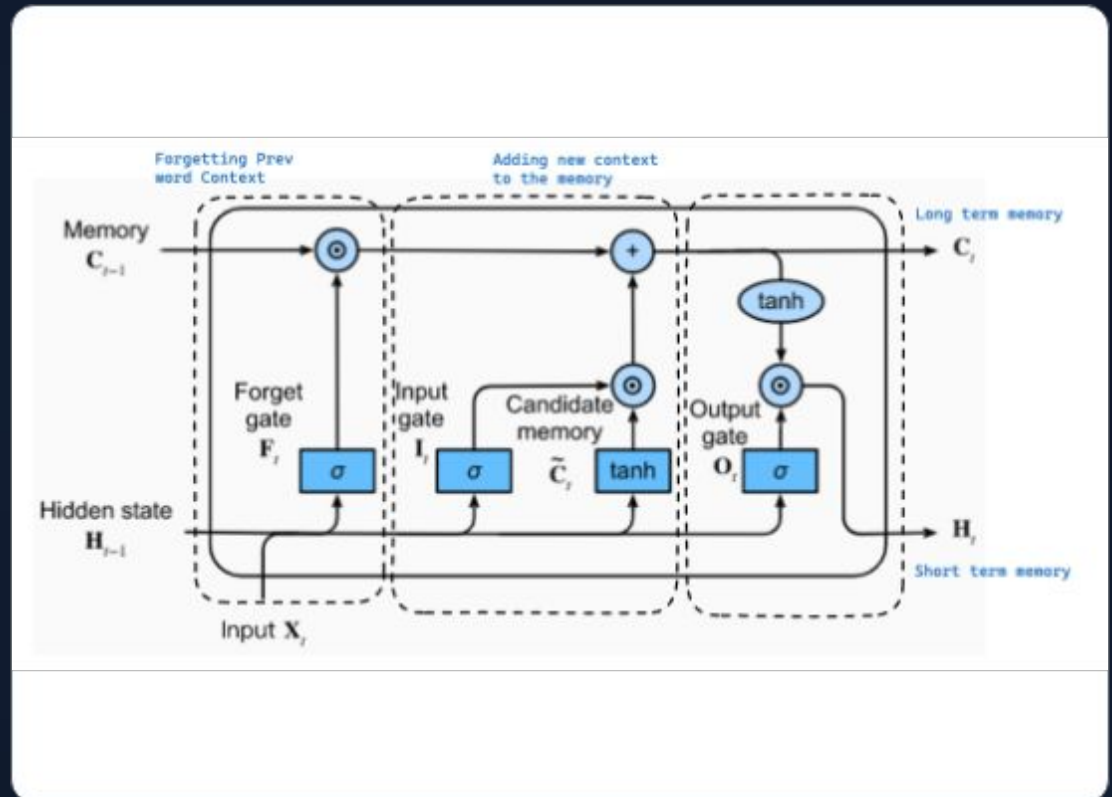
b_f: Bias term.

Step 2: The Input Gate

"What new info do we add?"

This step has two parts:

1. A **sigmoid layer** called the "input gate layer" decides which values we'll update.
2. A **tanh layer** creates a vector of new candidate values $\tilde{C}_t$ that could be added to the state.



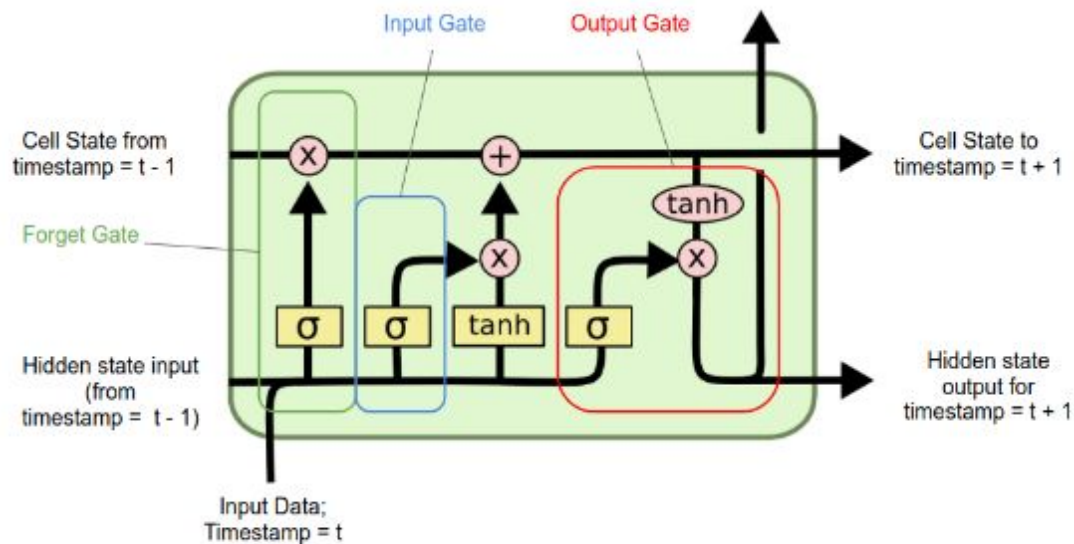
Input Gate Equations

$$i_t = \sigma W_i \cdot h_{t-1} x_t + b_i$$

$$\tilde{C}_t = \tanh W_C \cdot h_{t-1} x_t + b_C$$

`i_t` decides **which** values to update. `c_t` are the **new candidate** values.

Step 3: Updating the Cell State



The Actual Update

We multiply the old state by (forgetting things) f_t and add (adding new scaled candidates).

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

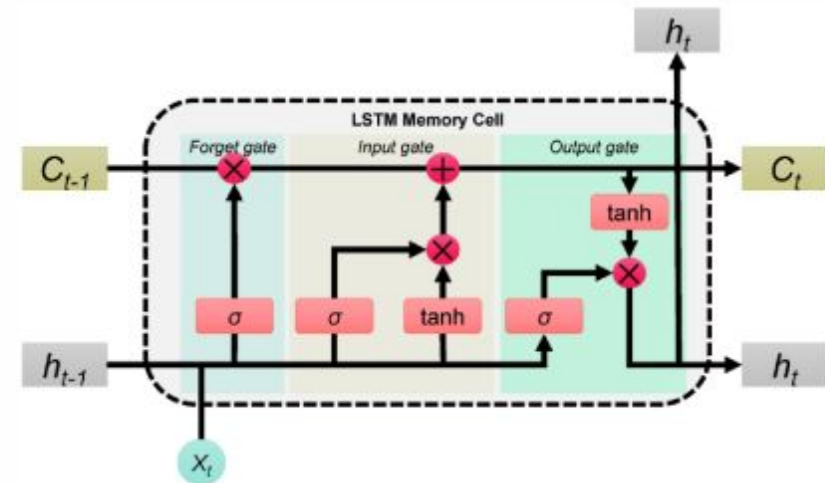
This is where the magic happens: dropping old info and storing new info.

Step 4: The Output Gate

"What do we reveal?"

The output is a filtered version of our cell state.

1. A sigmoid layer decides which parts of the cell state we are going to output σ_t
2. We put the cell state through **tanh** (to push values between -1 and 1).
3. Multiply it by the sigmoid gate output.



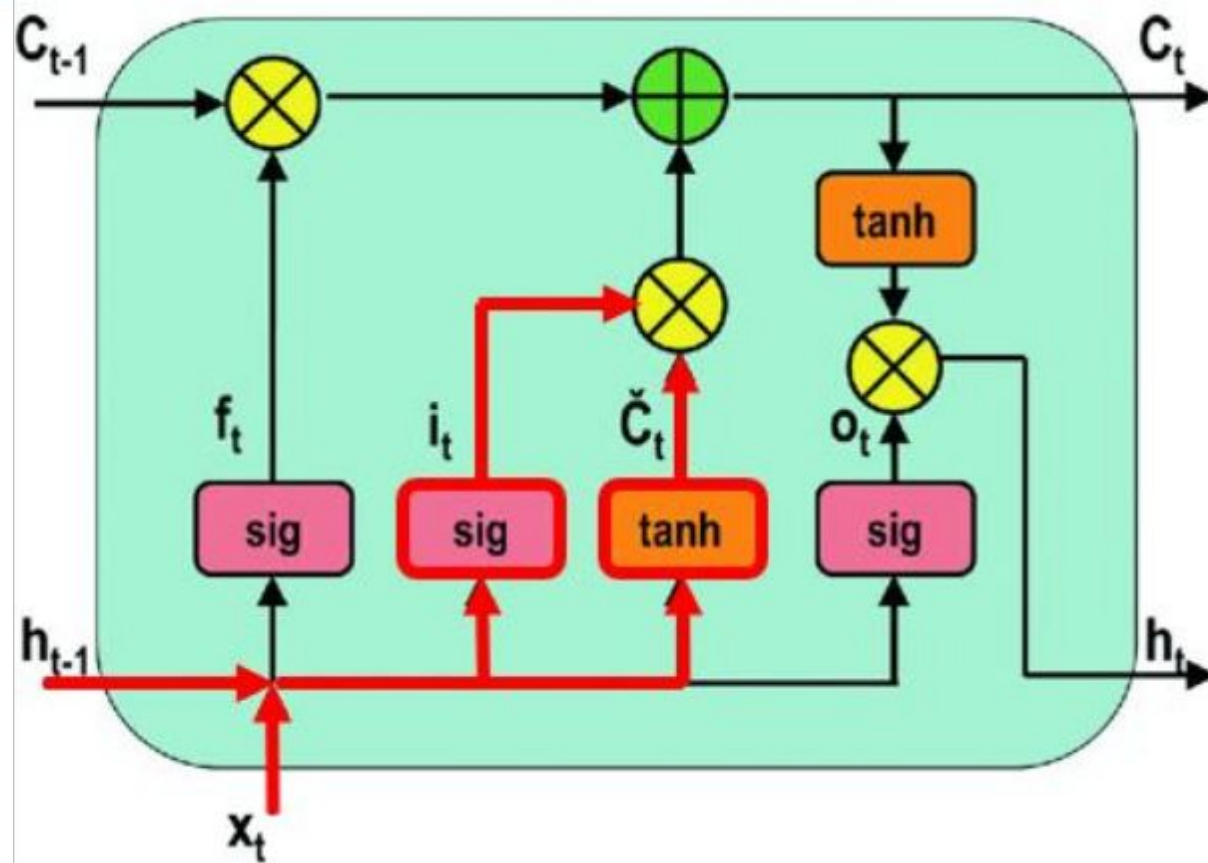
Output Gate Equations

$$o_t = \sigma W_o \cdot h_{t-1} x_t + b_o$$

$$h_t = o_t * \tanh C_t$$

h_t is the new hidden state that is passed to the next time step (and potentially to the output layer of the network).

Visual Summary: The Complete Cell



Input Gate Operation

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

$t = \text{timestep}$

$i_t = \text{input gate at } t$

$W_i = \text{Weight matrix of sigmoid operator between input gate and output gate}$

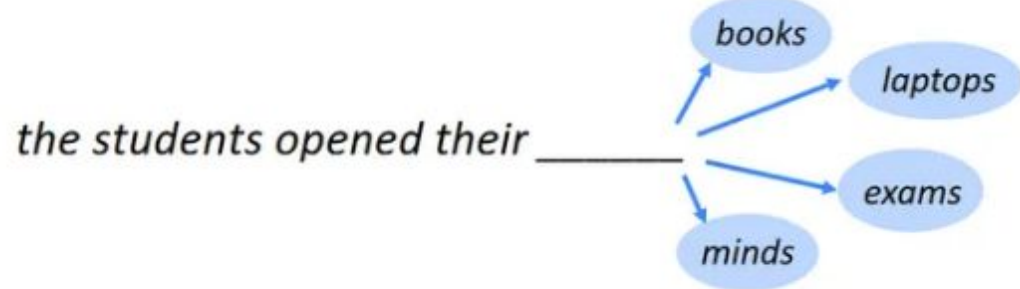
$b_t = \text{bias vector at } t$

$\tilde{C}_t = \text{value generated by tanh}$

$W_C = \text{Weight matrix of tanh operator between cell state information and network output}$

$b_C = \text{bias vector at } t, \text{ w.r.t } W_C$

9. Example: Language Modeling



Task: Next Word Prediction

We want to predict the next word based on previous context.

Context:
"The clouds are in the..."

Target:
"sky"

Example Step 1: Input

Input Vector (x_t)

The word "the" is converted into a vector (e.g., Word2Vec embedding).



Previous Hidden State (h_{t-1})

Contains information about the previous words: "The clouds are in".

Example Step 2: Forget Gate

Imagine the cell state currently holds the gender of a subject from a previous sentence (e.g., "Alice...").

But now we are talking about "clouds" (plural, neutral).

Action

The forget gate sees "clouds" and decides to **forget** the gender information of "Alice".

Forget Gate Output ≈ 0 (for gender dimensions)

Example Step 3: Input Gate

We need to store new information about the current subject.

Current Subject: "clouds".

Action

The input gate decides to **add** the information that the subject is "plural" to the cell state.

Input Gate Output ≈ 1 (for plurality dimensions)

Example Step 4: Output

Cell State Updated

Old gender info removed. New "plural" info added.

Final Prediction

The Output Gate looks at the cell state (containing "plural" and "nature context").

It outputs a hidden state that, when passed to a Softmax layer, predicts:

"SKY"

10. LSTM Variants: Peephole Connections

Standard LSTM

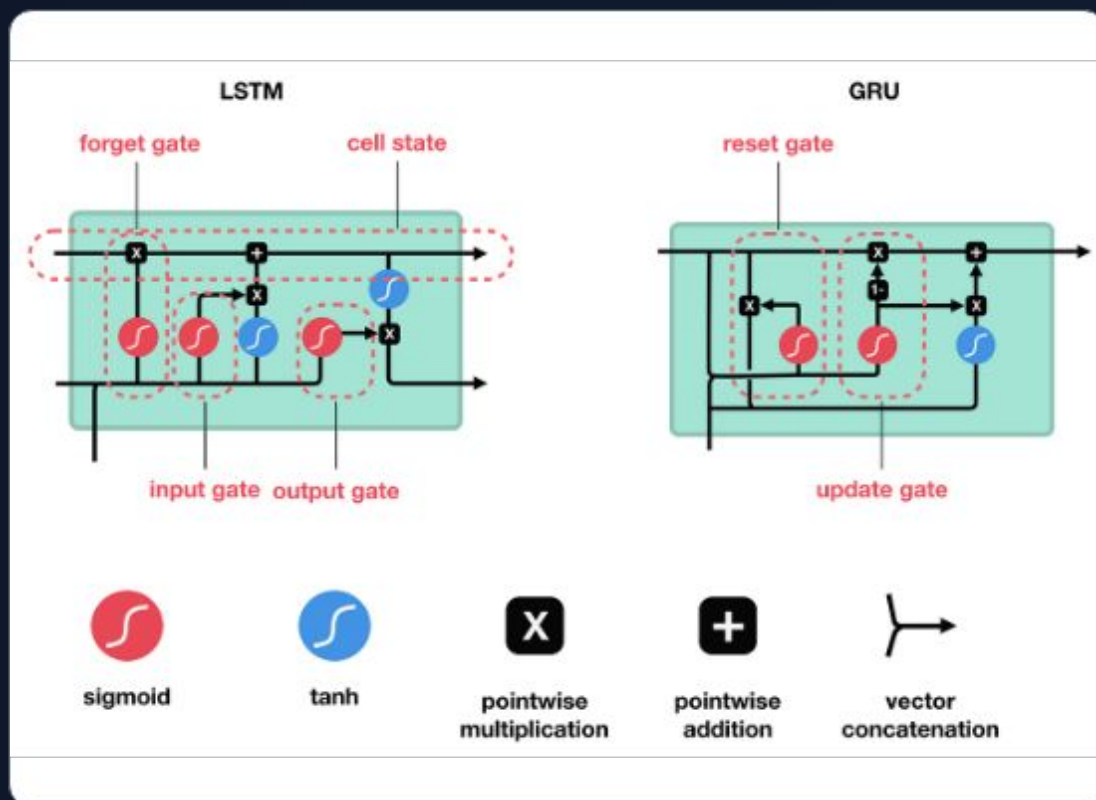
Gates only look at the previous hidden state (h_{t-1}) and input (x_t).

Peephole LSTM

We let the gate layers look at the **cell state** (c_{t-1}) as well.

This helps when the timing of the output depends heavily on the remaining cell state.

Gated Recurrent Unit (GRU)



A Simplified Cousin

Introduced by Cho et al. (2014).

- Combines Forget and Input gates into a single **"Update Gate"**.
- Merges Cell State and Hidden State.
- Simpler to compute, often performs just as well.

11. Real World Applications



Machine Translation

Google Translate (GNMT) uses LSTMs to map input sentences to output sentences.



Speech Recognition

Apple's Siri and Amazon Alexa use LSTMs to process audio streams.



Time Series

Predicting stock prices, electricity demand, or anomaly detection in sensors.

Pros & Cons

Advantages

- Solves Vanishing Gradient problem.
- Handles long-term dependencies effectively.
- Can handle variable length inputs.

Disadvantages

- Computationally expensive (slow training).
- Complex to tune (many parameters).
- Newer Transformer models (BERT, GPT) often outperform LSTMs in NLP.

Conclusion

Questions?

LSTMs revolutionized Deep Learning by giving networks **memory**.

While Transformers are the current trend in NLP, LSTMs remain a fundamental tool for Time Series and resource-constrained sequence tasks.

